

Terminology

Term	Description
lr, l (bit)	link register (special register)
pc, p (bit)	program counter (special register)
imm(n), i (bit)	immediate value (n bits)
condition, c (bit)	condition under which the instruction is executed. conditions are a combination of the status flags. see the chapter 'conditions' .
< >	parameters in angle brackets are optional
∈	is element of
⊆	subset or equal

SYNOPSIS
parameter value ranges

31/15.....0

 (machine code)

flags updated

instruction explanation in syntax similar to C

Move

mov rd, rm T1
rd, rm ∈ { r0..r11 }

010001

10

d

mmmm

ddd

rd = rm;

movs rd, rm T2
rd, rm ∈ { r0..r7 }

000

00

00000

mmm

ddd

N Z

rd = rm;

movs rd, #imm8 T1
rd ∈ { r0..r7 }
imm8 ∈ { 0..7 }

001

00

ddd

iiiiiii

N Z

rd = imm8;

Add

add rd, rm T2
rd, rm { r0..r11 }

010001

00

d

mmmm

ddd

rd = rd + rm;

adds rd, <rn>, rm T1
rd, rn, rm ∈ { r0..r7 }

000

11

0

0

mmm

nnn

ddd

N Z C V

rd = rn + rm; or rd += rm;

adcs rd, rm T1
rd, rm ∈ { r0..r7 }

010000

0101

mmm

ddd

N Z C V

rd = rd + rm + carry;

adds rd, rn, #imm3 T1
rd, rn ∈ { r0..r7 }
imm3 ∈ { 0..7 }

000

11

1

0

iii

nnn

ddd

N Z C V

rd = rn + imm3;

adds rd, #imm8 T1
rd ∈ { r0..r7 }
imm8 ∈ { 0..255 }

001

10

ddd

iiiiiii

N Z C V

rd = rd + imm8;

Subtract

subs rd, <rn>, rm T1
rd, rn, rm { r0..r7 }

000

11

0

1

mmm

nnn

ddd

N Z C V

rd = rn - rm; or rd -= rm;

rsbs rd, <rm> T1
rd, rm { r0..r7 }

010000

1001

nnn

ddd

N Z C V

rd = 0 - rn; or rd = 0 - rd;

sbc rd, rm T1
rd, rm { r0..r7 }

010000

0110

mmm

ddd

N Z C V

rd = rd - rm - (-carry);

subs rd, rn, #imm3 T1
rd, rn ∈ { r0..r7 }
imm3 ∈ { 0..7 }

000

11

1

1

iii

nnn

ddd

N Z C V

rd = rn - imm3; or rd -= imm3;

subs rd, #imm8 T2
rd ∈ { r0..r7 }
imm8 ∈ { 0..255 }

001

11

ddd

iiiiiii

N Z C V

rd = rd - imm8;

Multiply

0

1

0

0

0

1

1

0

1

n

n

n

d

d

d

N

Z

mul

s

rd, rn

rd, rn ∈ { r0..r7 }

rd = rd * rn;

T1

Compare

0

1

0

0

0

1

0

1

0

m

m

m

n

n

n

N

Z

C

V

cmp

rn, rm

rn, rm ∈ { r0..r7 }

update_flags(rn - rm);

T1

0

1

0

0

0

1

0

1

n

m

m

m

m

n

n

n

N

Z

C

V

cmp

rn, rm

rn, rm ∈ { r0..r11 }

update_flags(rn - rm);

T2

0

1

0

0

0

1

0

1

1

m

m

m

n

n

n

N

Z

C

V

cmn

rn, rm

rn, rm ∈ { r0..r7 }

update_flags(rn + rm);

T1

0

0

1

0

1

n

n

n

i

i

i

i

i

i

i

i

N

Z

C

V

cmp

rn, #imm8

rn ∈ { r0..r7 }

imm8 ∈ { 0..255 }

update_flags(rn - imm8);

T1

Logical

0

1

0

0

0

0

0

0

0

m

m

m

d

d

d

N

Z

ands

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd & rm;

T1

0

1

0

0

0

1

1

0

0

m

m

m

d

d

d

N

Z

orrs

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd | rm;

T1

0

1

0

0

0

0

0

0

1

m

m

m

d

d

d

N

Z

eors

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd ^ rm;

T1

0

1

0

0

0

1

1

1

0

m

m

m

d

d

d

N

Z

bics

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd & ~rm;

T1

0

1

0

0

0

1

1

1

1

m

m

m

d

d

d

N

Z

mvns

rd, rm

rd, rm ∈ { r0..r7 }

rd = ~rm;

T1

0

1

0

0

0

1

0

0

0

m

m

m

n

n

n

N

Z

tst

rn, rm

rn, rm ∈ { r0..r7 }

update_flags(rn & rm);

T1

Shift / Rotate

0

1

0

0

0

0

0

1

0

m

m

m

d

d

d

N

Z

C

lsls

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd << rm;

T1

0

1

0

0

0

0

0

1

1

m

m

m

d

d

d

N

Z

C

lsrs

rd, rm

rd, rm ∈ { r0..r7 }

rd = rd >> rm;

T1

0

1

0

0

0

0

1

0

0

m

m

m

d

d

d

N

Z

C

asrs

rd, rm

rd, rm ∈ { r0..r7 }

rd = extend_sign(rd >> rm);
(rd = rd / 2^{rm})

T1

0

0

0

0

0

i

i

i

i

i

m

m

m

d

d

d

N

Z

C

lsls

rd, <rm>, #imm5

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

rd = rm << imm5; or rd <=< imm5;

T1

0

0

0

0

1

i

i

i

i

i

m

m

m

d

d

d

N

Z

C

lsrs

rd, <rm>, #imm5

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

rd = rm >> imm5; or rd >=> imm5;

t1

0

0

0

1

0

i

i

i

i

i

m

m

m

d

d

d

N

Z

C

asrs

rd, <rm>, #imm5

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

rd = extend_sign(rm >> imm5);
or rd = extend_sign(rd >> imm5);
(rd = rm / 2^{imm5})

T1

0

1

0

0

0

0

1

1

1

m

m

m

d

d

d

N

Z

C

rors

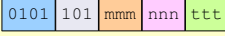
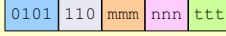
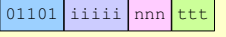
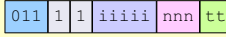
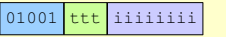
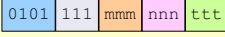
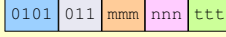
rd, rm

rd, rm ∈ { r0..r7 }

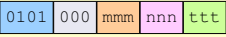
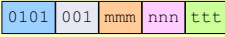
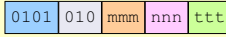
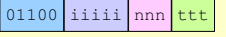
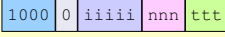
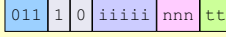
rd = rd >> rm | rd << (32 - rm);
(rotate right)

T1

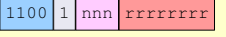
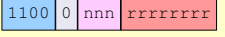
Load

ldr rt, [rn, rm] <i>T1</i> rt ∈ { r0..r7, pc, sp } rn, rm ∈ { r0..r7 }  rt = *((uint32_t)(rn + rm));	ldrh rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  rt = *((uint16_t)(rn + rm));	ldrb rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  rt = *((uint8_t)(rn + rm));
ldr rt, [rn, <#imm5>] <i>T1</i> rt ∈ { r0..r7, pc, sp } rn ∈ { r0..r7 } imm5 ∈ { 0..31 } << 2  rt = *((uint32_t)(rn + imm5 << 2));	ldrh rt, [rn, <#imm5>] <i>T1</i> rt, rn ∈ { r0..r7 } imm5 ∈ { 0..31 } << 1  rt = *((uint16_t)(rn + imm5 << 1));	ldrb rt, [rn, <#imm5>] <i>T1</i> rt, rn ∈ { r0..r7 } imm5 ∈ { 0..31 }  rt = *((uint8_t)(rn + imm5));
ldr rt, [#imm8] <i>T1</i> rt ∈ { r0..r7, pc, sp } imm8 ∈ { 0..255 } << 2  rd = *((uint32_t)(pc + imm8 << 2));	ldrsh rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  rt = extend_sign(((uint16_t)(rn + rm)));	ldrshb rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  rt = extend_sign(((uint8_t)(rn + rm)));

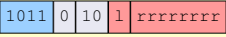
Store

str rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  *((uint32_t)(rn + rm)) = rt;	strh rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  *((uint16_t)(rn + rm)) = rt;	strb rt, [rn, rm] <i>T1</i> rt, rn, rm ∈ { r0..r7 }  *((uint8_t)(rn + rm)) = rt;
str rt, [rn, <#imm5>] <i>T1</i> rt, rn ∈ { r0..r7 } imm5 ∈ { 0..31 } << 2  *((uint32_t)(rn + imm5 << 2)) = rt;	strh rt, [rn, <#imm5>] <i>T1</i> rt, rn ∈ { r0..r7 } imm5 ∈ { 0..31 } << 1  *((uint16_t)(rn + imm5 << 1)) = rt;	strb rt, [rn, <#imm5>] <i>T1</i> rt, rn ∈ { r0..r7 } imm5 ∈ { 0..31 }  *((uint8_t)(rn + imm5)) = rt;

Load / Store Multiple Registers

ldm rn<!,>, { registers } <i>T1</i> rn ∈ { r0..r7 } registers ⊆ { r0..r7 } <!> also load rn  for (k = 0; k < #regs; k++){ regs[k] = *((uint32_t)(rn + k)); } 	stm rn<!,>, { registers } <i>T1</i> rd, ∈ { r0..r7 } registers ⊆ { r0..r7 } <!> also load rn  for (k = 0; k < #regs; k++){ *((uint32_t)(rn + k)) = regs[k]; }
--	---

Push / Pop

push { registers } <i>T1</i> registers ⊆ { r0..r7, lr }  for (k = 0; k < #regs; k++){ stack[k] = regs[k]; } 	pop { registers } <i>T1</i> registers ⊆ { r0..r7, pc }  for (k = 0; k < #regs; k++){ regs[k] = stack[k]; }
---	--

Conditions

This table lists the conditions to be used with conditional instructions. In CT1 we use only one instruction with a condition, the [branch with label T1 encoded](#). Note that the conditions have distinct meaning, depending on the operands being unsigned or signed.

bits	symbol	flags	context	description
0000	eq	Z == 1	all	equal
0001	ne	Z == 0	all	not equal
0010	cs / hs	C == 1	unsigned	carry set / higher than or same
0011	cc / lo	C == 0	unsigned	carry cleared / lower than
0100	mi	N == 1	signed	minus / negative
0101	pl	N == 0	signed	plus / positive or zero
0110	vs	V == 1	signed	overflow set (overflow)
0111	vc	V == 0	signed	overflow cleared (no overflow)
1000	hi	C == 1 && Z == 0	unsigned	higher than
1001	ls	C == 0 Z == 1	unsigned	lower than or same
1010	ge	N == V	signed	greater than or equal
1011	lt	N != V	signed	less than
1100	gt	Z == 0 && N == V	signed	greater than
1101	le	Z == 1 N != V	signed	less than or equal
1110	al		all	always == no condition
1111	-			-

Flag Dependent

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
cs	carry set	C == 1
cc	carry cleared	C == 0
mi	minus / negative	N == 1
pl	plus / positive or zero	N == 0
vs	overflow set (overflow)	V == 1
vc	overflow cleared (no overflow)	V == 0

Arithmetic Unsigned

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
hs (cs)	unsigned higher than or same	C == 1
lo (cc)	unsigned lower than	C == 0
hi	higher than	C == 1 && Z == 0
ls	lower than or same	C == 0 Z == 1

Arithmetic Signed

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
mi	minus / negative	N == 1
pl	plus / positive or zero	N == 0
vs	overflow set (overflow)	V == 1
vc	overflow cleared (no overflow)	V == 0
ge	signed greater than or equal	N == V
lt	signed less than	N != V
gt	signed greater than	Z == 0 && N == V
le	signed less than or equal	Z == 1 N != V

Disassembly Table

[0000 / 0x0]		
0000 0000 00mm mddd	movs rd, rm	// rd = rm (alias for 'lsls rd, rm, #0')
0000 0iii iimm mddd	lsls rd, rm, #imm5	// rd = rm << imm5
0000 1iii iimm mddd	lsrs rd, rm, #imm5	// rd = rm >> imm5
[0001 / 0x1]		
0001 0iii iimm mddd	asrs rd, rm, #imm5	// rd = rm << imm5 (keep sign)
0001 100m mnnn nddd	adds rd, rn, rm	// rd = rn + rm
0001 101m mnnn nddd	subs rd, rn, rm	// rd = rn - rm
0001 110i iinn nddd	adds rd, rn, #imm5	// rd = rn + imm5
0001 111i iinn nddd	subs rd, rn, #imm5	// rd = rn - imm5
[0010 / 0x2]		
0010 0ddd iiii iiii	movs rd, #imm8	// rd = #imm8
0010 1nnn iiii iiii	cmp rn, #imm8	// flags = rn - imm8
[0011 / 0x3]		
0011 0ddd iiii iiii	adds rd, #imm8	// rd = rd + imm8
0011 1ddd iiii iiii	subs rd, #imm8	// rd = rd - imm8
[0100 / 0x4]		
0100 0000 00mm mddd	ands rd, rm	// rd = rd & rm
0100 0000 01mm mddd	eors rd, rm	// rd = rd ^ rm
0100 0000 10mm mddd	lsls rd, rm	// rd = rd << rm
0100 0000 11mm mddd	lsrs rd, rm	// rd = rd >> rm
0100 0001 00mm mddd	asrs rd, rm	// rd = rd >> rm (keep sign)
0100 0001 01mm mddd	adcs rd, rm	// rd = rd + rm + carry
0100 0001 10mm mddd	sbcas rd, rm	// rd = rd - rm - (-carry)
0100 0001 11mm mddd	rors rd, rm	// rd = rd << (32 - rm) rd >> rm
0100 0010 00mm mddd	tst rd, rm	// flags = rd & rm
0100 0010 01mm mddd	rsbs rd, rm, #0	// rd = 0 - rm ('neg' is an alias to 'rsb')
0100 0010 10mm mnnn	cmp rn, rm	// flags = rn - rm
0100 0010 11mm mnnn	cmn rn, rm	// flags = rn + rm
0100 0011 00mm mddd	orrs rd, rm	// rd = rd rm
0100 0011 01mm mddd	mulas rd, rm, rd	// rd = rd * rm
0100 0011 10mm mddd	bicas rd, rm	// rd = rd & ~rm (bit clear)
0100 0011 11mm mddd	mvns rd, rm	// rd = ~rm
0100 0100 dmmmm mddd	add rd, rm	// rd = rd + rm
0100 0101 nmnmnn mnnn	cmp rn, rm	// flags = rn - rm
0100 0110 dmmmm mddd	mov rd, rm	// rd = rm
0100 0111 0mmmm m...	bx rm	// pc = rm
0100 0111 1mmmm m...	blx rm	// lr = pc lu, pc = rm
0100 1ttt iiii iiii	ldr rt, [pc, #imm11]	// rt = *((uint32_t*)(pc % 4 + (imm11 << 2)))
	ldr rt, =label	// label/literal is stored in the literal
	ldr rt, =literal	// pool and loaded via an offset from the PC
[0101 / 0x5]		
0101 000m mnnn nttt	str rt, [rn, rm]	// *((uint32_t*)(rn + rm)) = rt
0101 001m mnnn nttt	strh rt, [rn, rm]	// *((uint16_t*)(rn + rm)) = rt
0101 010m mnnn nttt	strb rt, [rn, rm]	// *((uint8_t*)(rn + rm)) = rt
0101 011m mnnn nttt	ldrsb rt, [rn, rm]	// rt = *((int8_t*)(rn + rm))
0101 100m mnnn nttt	ldr rt, [rn, rm]	// rt = *((uint32_t*)(rn + rm))
0101 101m mnnn nttt	ldrh rt, [rn, rm]	// rt = *((uint16_t*)(rn + rm))
0101 110m mnnn nttt	ldrb rt, [rn, rm]	// rt = *((uint8_t*)(rn + rm))
0101 111m mnnn nttt	ldrsh rt, [rn, rm]	// rt = *((int16_t*)(rn + rm))
[0110 / 0x6]		
0110 0iii iinn nttt	str rt, [rn, #imm5]	// *((uint32_t*)(rn + (imm5 << 2))) = rt
0110 1iii iinn nttt	ldr rt, [rn, #imm5]	// rt = *((uint32_t*)(rn + (imm5 << 2)))
[0111 / 0x7]		
0111 0iii iinn nttt	strb rt, [rn, #imm5]	// *((uint8_t*)(rn + imm5)) = rt
0111 1iii iinn nttt	ldrb rt, [rn, #imm5]	// rt = *((uint8_t*)(rn + imm5))
[1000 / 0x8]		
1000 0iii iinn nttt	strh rt, [rn, #imm5]	// *((uint16_t*)(rn + (imm5 << 1))) = rt
1000 1iii iinn nttt	ldrh rt, [rn, #imm5]	// rt = *((uint16_t*)(rn + (imm5 << 1)))

```

[ 1001 / 0x9 ]
1001 0ttt iiii iiii    str rt, [sp, #imm8]    // *((uint32_t*)(sp + (imm8 << 2))) = rt
1001 1ttt iiii iiii    ldr rt, [sp, #imm8]    // rt = *((uint32_t*)(sp + (imm8 << 2)))

[ 1010 / 0xA ]
1010 0ddd iiii iiii    adr rd, label        // rd = (pc % 4) + (imm8 << 2)
1010 1ddd iiii iiii    add rd, sp, #imm8    // rd = sp + (imm8 << 2)

[ 1011 / 0xB ]
1011 0000 0iii iiii    add sp, sp, #imm7    // sp = sp + (imm7 << 2)
1011 0000 1iii iiii    sub sp, sp, #imm7    // sp = sp - (imm7 << 2)
1011 001i iiii innn    cbz rn, label        // 'armv7-m' and later only!
1011 0010 00mm mddd    sxth rd, rm        // rd = extend_sign((int16_t)rm)
1011 0010 01mm mddd    sxtb rd, rm        // rd = extend_sign((int8_t)rm)
1011 0010 10mm mddd    uxth rd, rm        // rd = (uint16_t)rm
1011 0010 11mm mddd    uxtb rd, rm        // rd = (uint8_t)rm
1011 0101 rrrr rrrr    push {lr, r0..r7}    // push selected registers to the stack (l = lr)
1011 0110 0100 ....    -                    // unpredictable
1011 0110 0101 0...    setend le            // armv7-a/r only!
1011 0110 0101 1...    setend be            // armv7-a/r only!
1011 0110 0110 0aif    cpsie <i><f>        // enable processor state i=irq, f=fiq
1011 0110 0111 0aif    cpsid <i><f>        // disable processor state i=irq, f=fiq
1011 0110 011x 1...    -                    // unpredictable
1011 101i iiii innn    cbnz rn, label        // 'armv7-m' and later only!
1011 1010 00mm mddd    rev rd, rm        // rd[31:0] = rm[7:0, 15:8, 23:16, 31:24]
1011 1010 01mm mddd    rev16 rd, rm        // rd[31:0] = rm[23:16, 31:24, 7:0, 15:8]
1011 1010 10.. ....    -                    // undefined
1011 1010 11mm mddd    revsh rd, rm        // rd[31:0] = extend_sign((int32_t)rm[7:0, 15:8])
1011 110p rrrr rrrr    pop {pc, r0..r7}    // pop selected registers from the stack (p = pc)
1011 1110 iiii iiii    bkpt #imm8        // breakpoint, arg ignored by hw
1011 1111 0000 0000    nop                // do nothing
1011 1111 0001 0000    yield                // signal to hw to suspend/resume threads
1011 1111 0010 0000    wfe                // wait for event
1011 1111 0011 0000    wfi                // wait for interrupt
1011 1111 0100 0000    sev                // signal event to multi-processor system
1011 1111 cccc mmmm    itssel cond        // 'armv7-m' and later only!

[ 1100 / 0xC ]
1100 0nnn rrrr rrrr    stmia rn<!> {reg0-7} // store selected registers at address in 'rn'
1100 1nnn rrrr rrrr    ldmia rn<!> {reg0-7} // load selected registers from address in 'rn'

[ 1101 / 0xD ]
1101 cccc iiii iiii    b<cond> label        // if (cond) goto label
1101 1110 .... ....    -                    // undefined

[ 1110 / 0xE ]
1110 0iii iiii iiii    b label            // goto label
1110 1... .... ....    -                    // 'armv7-m' and later only!

[ 1111 / 0xF ]
1111 0sii iiii iiii 11j1 kiii iiii iiii    bl label        // lr = pc | 1u, goto label
1111 001i 1110 1111 1000 dddd ssss ssss    mrs rd, rs        // rd = rs (rs = special register)
1111 001i 1000 mmmm 1000 1000 ssss ssss    msr rs, rm        // rs = rm (rs = special register)
1111 001i 1011 1111 1000 1111 0100 1111    dsb                // data synchronisation barrier
1111 001i 1011 1111 1000 1111 0101 1111    dmb                // data memory barrier
1111 001i 1011 1111 1000 1111 0110 1111    isb                // instruction synchronisation barrier

```

Status Flags

The N, Z, C, and V condition flags are held in the APSR.

flag	description
N	Set to 1 when the result of the operation is negative, cleared to 0 otherwise.
Z	Set to 1 when the result of the operation is zero, cleared to 0 otherwise.
C	Set to 1 when the operation results in a carry, or when a subtraction results in no borrow, cleared to 0 otherwise.
V	Set to 1 when the operation causes overflow, cleared to 0 otherwise.

Carry is set in the following situations:

- For an addition, including the comparison instruction **CMN**, C is set to 1 if the addition produced a carry (that is, an unsigned overflow), and to 0 otherwise.
- For a subtraction, including the comparison instruction **CMP**, C is set to 0 if the subtraction produced a borrow (that is, an unsigned underflow), and to 1 otherwise.
- For non-addition/subtractions that incorporate a shift operation, C is set to the last bit shifted out of the value by the shifter.

Overflow is set when the result of a signed add, subtract, or compare is greater than or equal to 2^{31} , or less than -2^{31} . Or in other words, when the sign would be different, if the operation was executed at infinite resolution.